

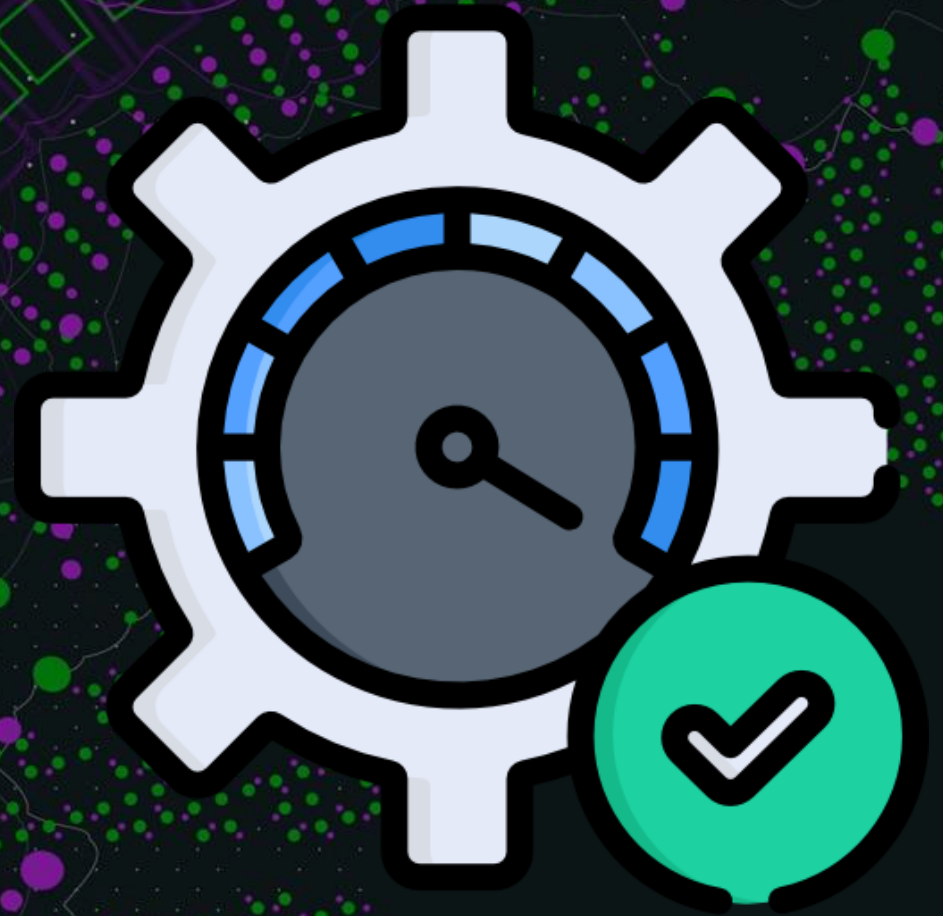
Pipeline Optimization With GitHub

Optimizing CI/CD pipeline

In the previous GitHub video...



In this video...



Current pipeline

```
# Workflow Name
name: Run CI/CD Tests

# Triggers
on:
  push:
    branches: [main, Section-2]
  pull_request:
    branches: [main, Section-2]

# Jobs
jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Get your code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Set up environment
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11.7"

      # Step 3: Install dependencies
      - name: Install dependencies
        run: python -m pip install --upgrade pip

      # Step 4: Install testing tools
      - name: Install pytest
        run: pip install pytest

      # Step 5: Run tests
      - name: Run tests
        run: pytest
```

```
on:
  push:
    branches: "**"
  pull_request:
    branches: "**"
```

Issues with a long pipeline in one job

- If one step fails:
 - The rest of the steps don't execute
- A long pipeline will take much longer to run
- A long pipeline will eat up resources much quicker

How to optimize a pipeline

- There are multiple ways but the main 2:
 1. Group the steps according to the different tests in its own job:
 - Backend tests (unit)
 - Frontend tests (unit, integration, e2e)
 2. Use caching:
 - Apply caching when installing dependencies

Using step 1



```
# 3. JOBS (WHAT IT DOES)
jobs:
  <job-id>:
    runs-on: <operating-system> # Ubuntu, Microsoft, etc

    steps:
      # Step 1: Get your code
      # Step 2: Set up environment
      # Step 3: Install dependencies
      # Step 4: Install testing tools
      # Step 5: Run tests

      # Each step must have a name:
      - name: <name>

      # Must be followed with a combination of:
      uses: <action-name>
      with:
        <key>: <value>
      run: <command>
```

Using step 2



Cache's are defined as a step within a job where needed

- **name:** <name-of-step>

- uses:** actions/cache@v4 # Cache version to use

- with:**

- path:** ~/.cache/ms-playwright # Path that GitHub uses to find and store it

- key:** \${{ runner.os }}-playwright # Name of the cache

Combining



```
backend-unit-tests:
  runs-on: ubuntu-latest

  steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11.7"

  - name: Install backend dependencies
    run: |
      python -m pip install --upgrade pip
      pip install flask flask-cors pytest

  - name: Run backend tests
    run: pytest
```



```
frontend-unit-tests:
  runs-on: ubuntu-latest

  steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up node
    uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: "npm"
      cache-dependency-path: app/frontend/package-lock.json

  - name: Install frontend dependencies
    working-directory: ./app/frontend
    run: npm install

  - name: Run frontend tests
    working-directory: ./app/frontend
    run: npm run test:unit
```

Combining

```
integration-tests:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.11.7"

    - name: Install backend dependencies
      run: |
        python -m pip install --upgrade pip
        pip install flask flask-cors

    - name: Start backend server
      working-directory: ./app
      run: |
        nohup python -m backend.app &
        sleep 5

    - name: Set up node
      uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "npm"
        cache-dependency-path: app/frontend/package-lock.json

    - name: Install frontend dependencies
      working-directory: ./app/frontend
      run: npm install

    - name: Run integration tests
      working-directory: ./app/frontend
      run: npm run test:integration
```

```
e2e-tests:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.11.7"

    - name: Install backend dependencies
      run: |
        python -m pip install --upgrade pip
        pip install flask flask-cors

    - name: Set up node
      uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: "npm"
        cache-dependency-path: app/frontend/package-lock.json

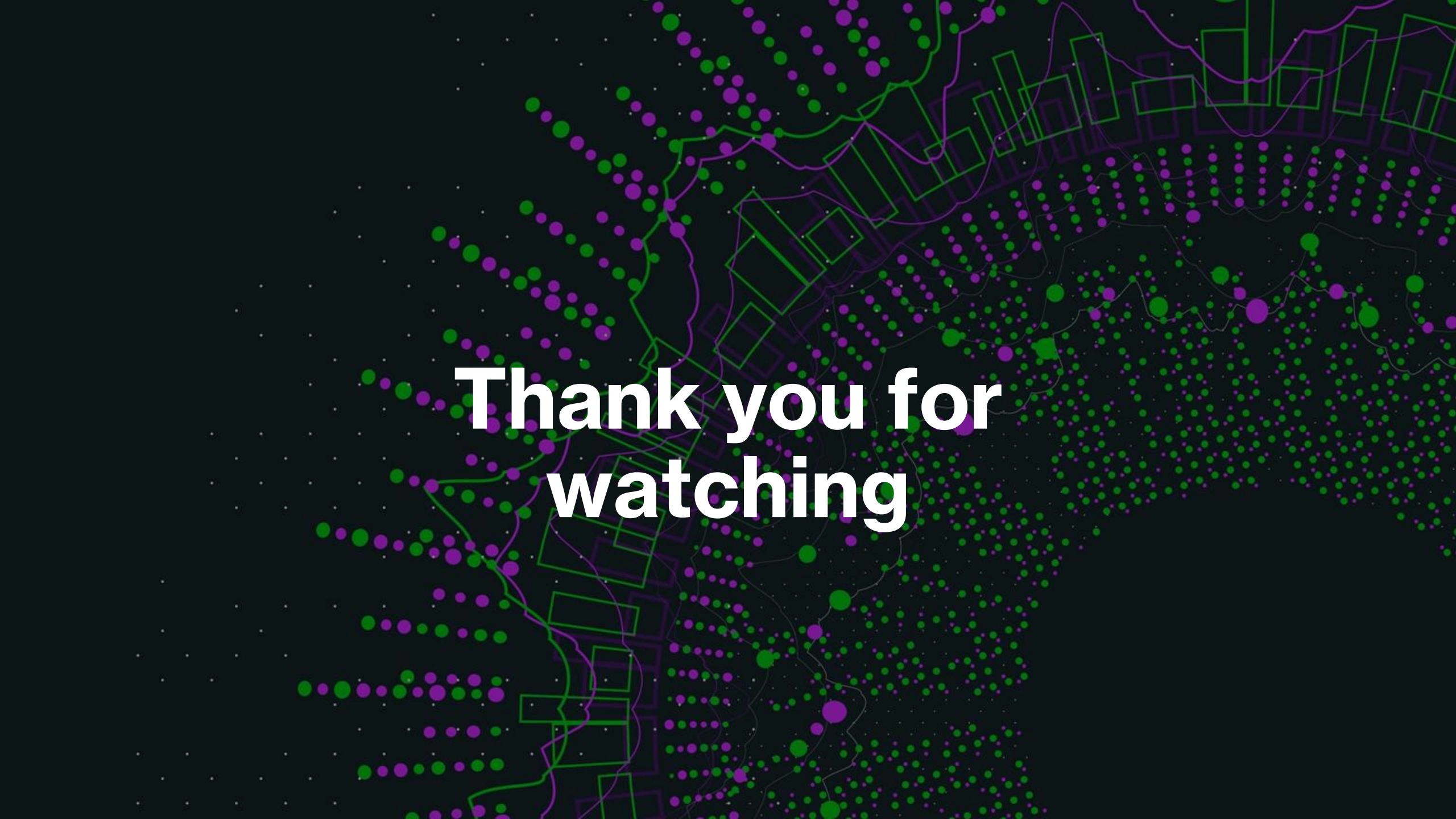
    - name: Install frontend dependencies
      working-directory: ./app/frontend
      run: npm install

    - name: Cache Playwright browsers
      uses: actions/cache@v4
      with:
        path: ~/.cache/ms-playwright
        key: ${runner.os}-playwright

    - name: Install playwright dependencies
      working-directory: ./app/frontend
      run: npx playwright install

    - name: Run e2e tests
      working-directory: ./app/frontend
      run: npm run test:e2e
```

Live Demo

The background is a black field filled with a complex pattern of green and purple dots, lines, and rectangles. The dots are arranged in various clusters and trails. The lines are thin and wavy, connecting some of the dots. The rectangles are thin and appear to be floating or overlapping the other elements. The overall effect is a dense, abstract, and somewhat chaotic visual field.

**Thank you for
watching**